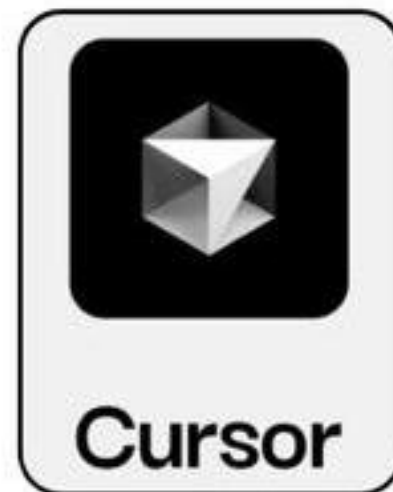
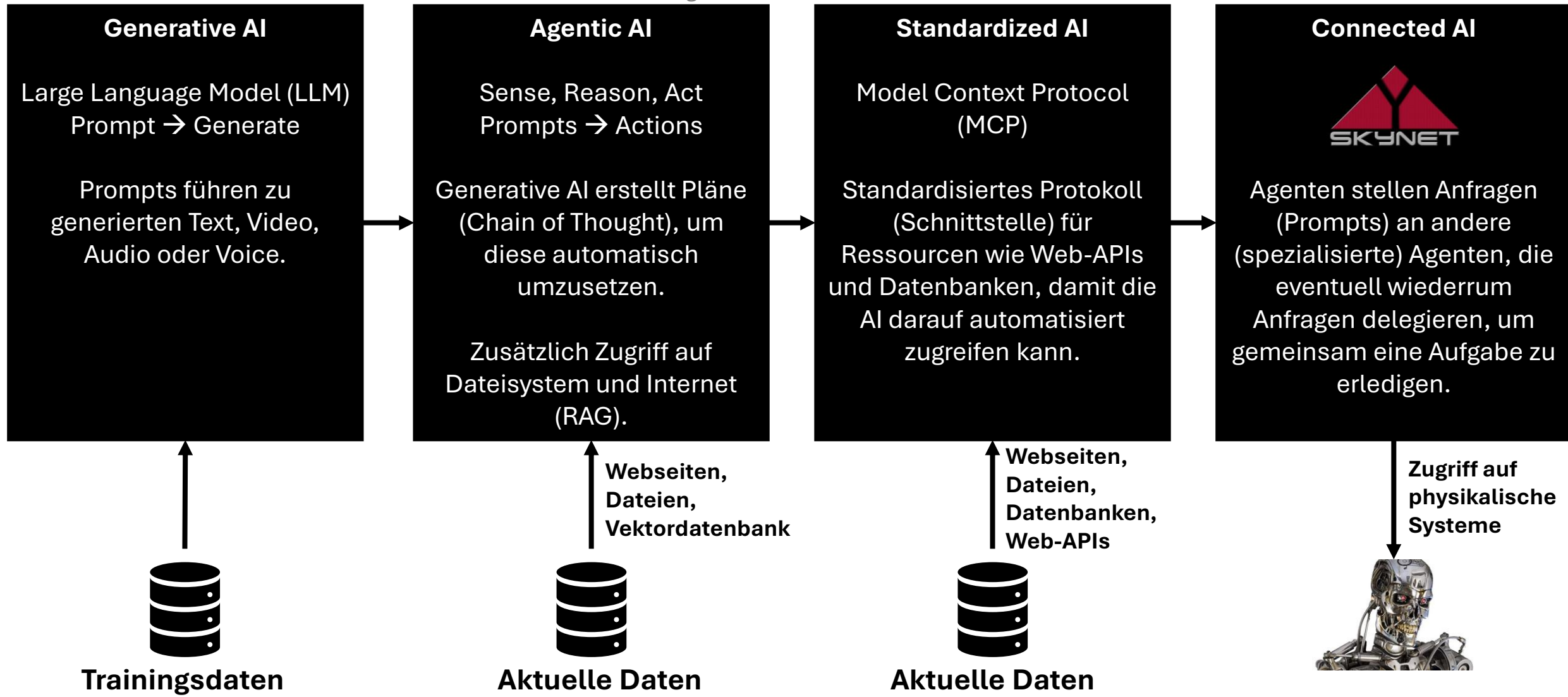


Vibe Coding



Entwicklung von AI

Basis für Vibe Coding



Vibe Coding: Definition



Andrej Karpathy ✓

@karpathy



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

Andrej Karpathy

Director of AI at Tesla.

Research Scientist and
founding member at OpenAI.

<https://karpathy.ai/>

Vibe Coding: Beispiel mit VSCode

```
src > App.vue > {} script setup
1  <script setup>
2  import { ref, computed, onMounted } from 'vue'
3  import draggable from 'vuedraggable'
4
5  // State
6  const newTask = ref('')
7  const tasks = ref([])
8  const filter = ref('all') // all, active, completed
9
10 // Load tasks from localStorage on mount
11 onMounted(() => {
12   const savedTasks = localStorage.getItem('todo-tasks')
13   if (savedTasks) {
14     tasks.value = JSON.parse(savedTasks)
15   } else {
16     // Add sample tasks if no tasks exist
17     tasks.value = [
18       {
```

Keep Undo 1 of 4 ↑ ↓

OUTPUT DEBUG CONSOLE TERMINAL PORTS DBCODE

```
npm install vuedraggable@next
oliviamcvicker@olivias-MacBook-Pro todo-app % npm install vuedraggable@next

added 1 package, and audited 51 packages in 678ms

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
oliviamcvicker@olivias-MacBook-Pro todo-app %
```

CHAT

olguzzar

add the ability to reorder tasks by dragging

App.vue:13-41

GitHub Copilot

> Used 1 reference

I'll help you add drag and drop functionality using vue-draggable. First, we need to install the required package.

✓ npm install vuedraggable@next

Now I'll modify the App.vue file to add drag and drop functionality to the task list.

App.vue +21 -3

Working...

1 file changed

App.vue src

34

Add Context...

App.vue Current file

Edit files in your workspace in agent mode

🎤

Agent

Claude 3.5 Sonnet

||

⌵

main* 0 0

Ln 3, Col 1 Spaces: 2 UTF-8 LF vue

Regeln für Vibe Coding



Daumenregeln:

1. Verwende einen weit verbreiteten Technology-Stack (Frameworks, Bibliotheken, Datenbank).
2. Übergebe ein Dokument mit Hinweisen an die AI (z. B. Guidelines, Best Practices), die sie kontinuierlich befolgt.
3. Die Aufgaben- bzw. Anforderungsbeschreibung an die AI sollte so präzise und detailliert wie möglich sein (um zu vermeiden, dass die AI kreativ wird).
4. Speichere (versioniere) funktionierenden Code mit Git.



1. **Start from a template:** Begin your project by cloning a template from GitHub or another source to provide a solid foundation. (On cursor, Start from Repo, and paste this link to build a nextjs app that is prebuilt with AI features, database and authorization <https://github.com/ansh/template-2>)
2. **Use agent mode:** Utilize Cursor's agent mode (not normal mode) to create, edit, and manage files through natural language commands.
3. **Use Perplexity:** Use perplexity to find new designs and api's from the web. Say that you are creating a nextjs project, and that you want to create ___x___ feature, and to give you instructions AND code examples.
4. **Create new chats in Composer:** Open a new Composer chat for each distinct task. Keep agent chats short.
5. **Run locally, test frequently:** Use built-in servers to run your app locally and test often to catch issues early.
6. **Iterate and refine:** Embrace rapid iteration—don't worry about perfect designs initially; improve them step by step.
7. **Utilize voice-to-text:** Use tools like *Whispr Flow* for faster input, and just vibe.
8. **Clone and fork wisely:** Use GitHub repos as starting templates to accelerate development, or to find inspiration, then customize them to fit your vision.
9. **Copy errors and paste into Composer agent:** When errors occur, copy error messages from your console and paste them into the Composer agent, and more times than not, it will be fixed. When dealing with errors, over explain the issue if it's not fixed the first time.
10. **Don't forget you can restore previous Composer chats:** Save your work frequently so you can revert to an earlier state if needed.
11. **Secure your secrets:** Always store API keys and sensitive data in environment files rather than hard-coding them.
12. **Commit often:** Push your progress to GitHub regularly to track changes and safeguard your work. Cursor can do this for you, just ask the agent.
13. **Deploy early:** Use platforms like Vercel to deploy your app early, to make sure there are no errors on deployment.
14. **Keep a record of the prompts that work best; reuse them often:** Document your most effective prompts to make future development and debugging easier.
15. **Enjoy the process—just vibe:** Embrace the creative journey of vibe coding, experiment, learn, and have fun along the way. just vibe.

Guidelines

Beispiel für Hinweise an die AI (z. B. Guidelines, Best Practices), die sie kontinuierlich befolgen soll.

BTW: Die kann man sich ebenfalls von AI generieren lassen. 😊

Best Practices for AstroJS, React, Tailwind CSS, and Nanostores

Application Structure

Project Organization

```
src/
├── components/
│   ├── ui/           # Reusable UI components
│   └── features/      # Feature-specific components
├── layouts/          # Astro layout components
├── pages/             # Astro pages (file-based routing)
├── stores/            # Nanostores state management
├── utils/             # Utility functions and helpers
├── styles/            # Global styles and Tailwind customizations
└── assets/            # Static assets like images, fonts
```

Feature Organization (for larger applications)

```
src/
├── features/
│   ├── auth/
│   │   ├── components/ # React components specific to auth
│   │   ├── stores/      # Auth-related nanostores
│   │   └── utils/       # Auth-specific utilities
│   └── products/
│       ├── components/
│       └── stores/
├── shared/           # Shared components, stores, utilities
│   ├── components/
│   └── utils/
└── pages/            # Astro pages using components from features
```

Component Creation Guidelines

When to Create a New Component

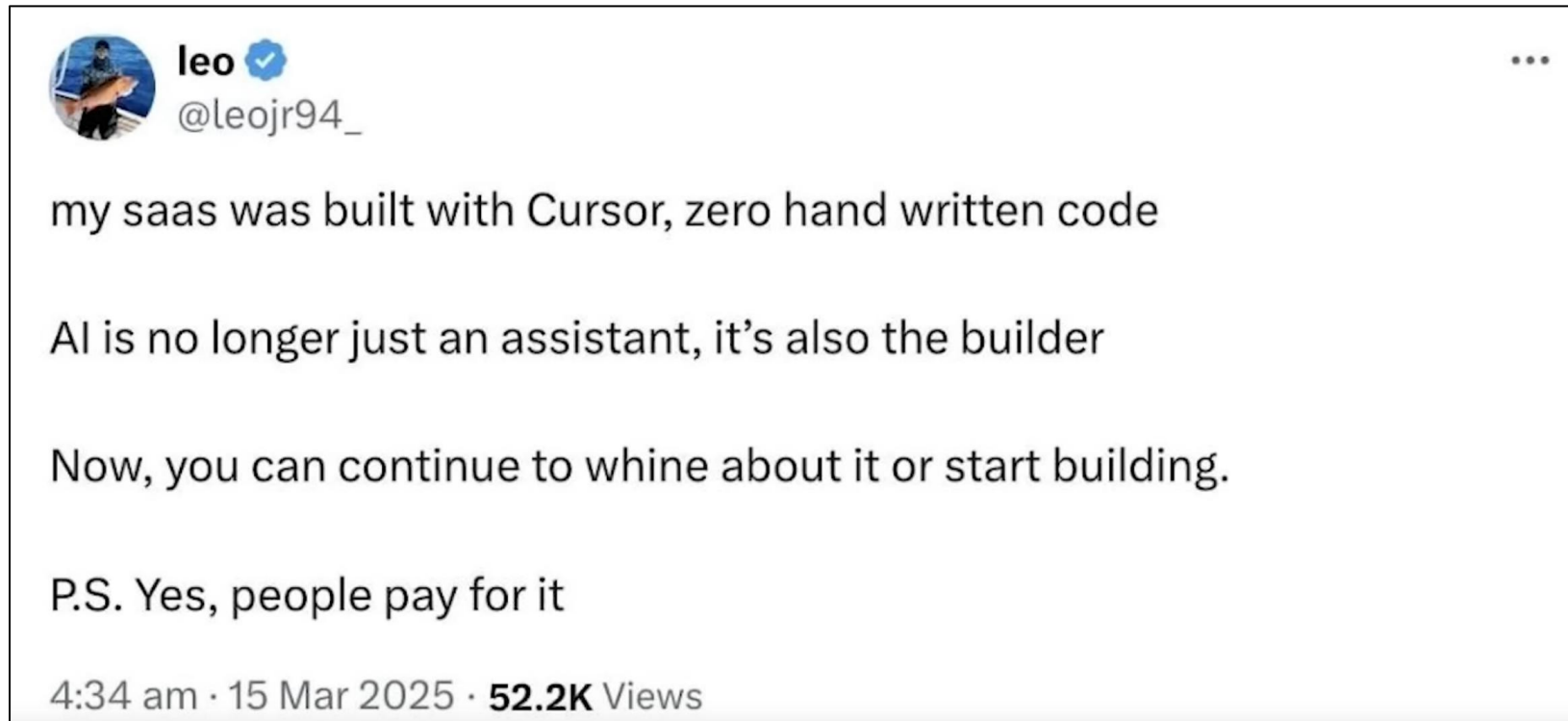
- Create a new component when:
 - UI element will be reused in multiple places
 - A section of UI exceeds 100 lines of code
 - A section has its own distinct responsibility or purpose
 - Logic for a section becomes complex (more than 3 state variables or hooks)
 - Component has a specific interaction pattern (dropdown, modal, etc.)

Anforderungen

Beispiel für ein Anforderungsdokument.

Requirement ID	Description	User Story	Expected Behavior/Outcome
FR001	Creating a New URL List	As a user, I want to be able to start a new, empty list so I can begin adding URLs.	The system should provide a clear way for the user to initiate the creation of a new list, potentially presenting an empty list view or an "add new list" button.
FR002	Adding URLs to a List	As a user, I want to be able to input or paste URLs into my list so I can compile the resources I want to share.	The system should provide an input field or mechanism for users to enter URLs. Upon submission, the URL should be added to the list.
FR003	Viewing URLs in a List	As a user, I want to be able to see all the URLs I have added to my list so I can review and manage them.	The system should display the list of added URLs, potentially with their automatically fetched titles and descriptions.
FR004	Editing URLs in a List	As a user, I want to be able to edit a URL in my list if I made a mistake or if the link has changed.	The system should provide a way for users to modify the URL of an existing item in the list.
FR005	Deleting URLs from a List	As a user, I want to be able to remove a URL from my list if it's no longer relevant or if I added it by mistake.	The system should provide a mechanism for users to delete individual URLs from their list.
FR006	Customizing the List URL	As a user, I want to be able to choose a custom URL for my list so it's easy to remember and share.	The system should provide an input field where users can specify their desired custom URL. It should also handle cases where the chosen URL is already taken.
FR007	Automatic URL Generation	As a user, if I don't want to think of a custom URL, I want the system to automatically create one for my list so I can still share it.	The system should generate a unique and shareable URL for the list if the user doesn't provide one.
FR008	Publishing a List	As a user, I want to be able to publish my list so that others can view the collection of URLs.	Upon publishing, the list should be saved and made publicly accessible at the associated URL.
FR009	Sharing a List	As a user, I want to be able to easily share the link to my list with others through various channels.	The system should provide the user with the URL of their list, potentially with options for copying it.
FR010	Accessing a Shared List	As a recipient, I want to be able to view the collection of URLs by clicking on or entering the shared link.	When a user visits a The Urlist link, they should be presented with the list of URLs.
FR011	Viewing all lists	As a user, I want to be able to see all the lists I have created so I can manage them easily.	The system should provide a dashboard or overview page where users can see all their created lists.
FR012	Deleting a List	As a user, I want to be able to delete an entire list if I no longer need it.	The system should provide a way for users to delete their entire list, including all associated URLs.

Vibe Coding: Praxisbeispiel @leojr94_ 1/2

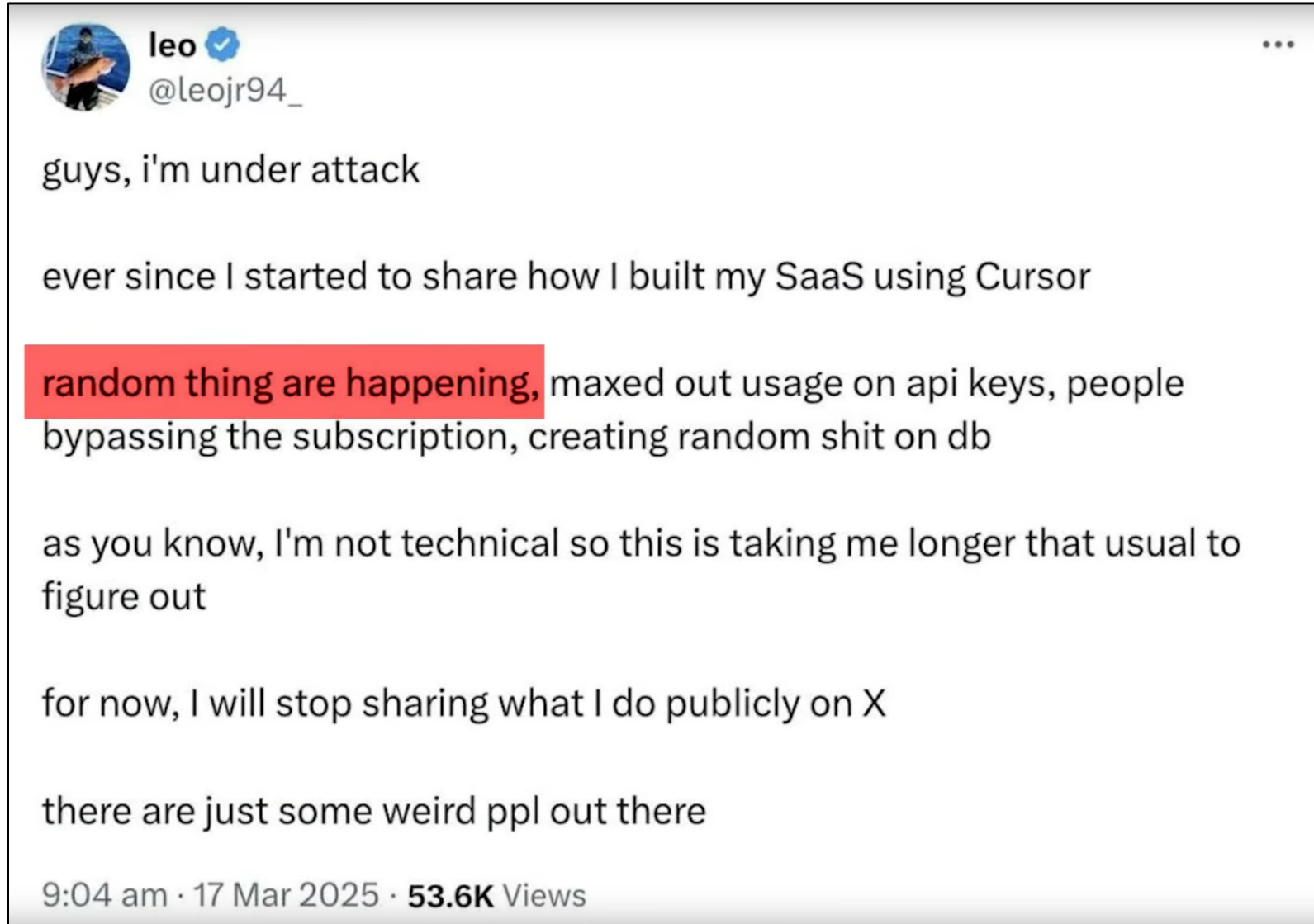


SaaS:

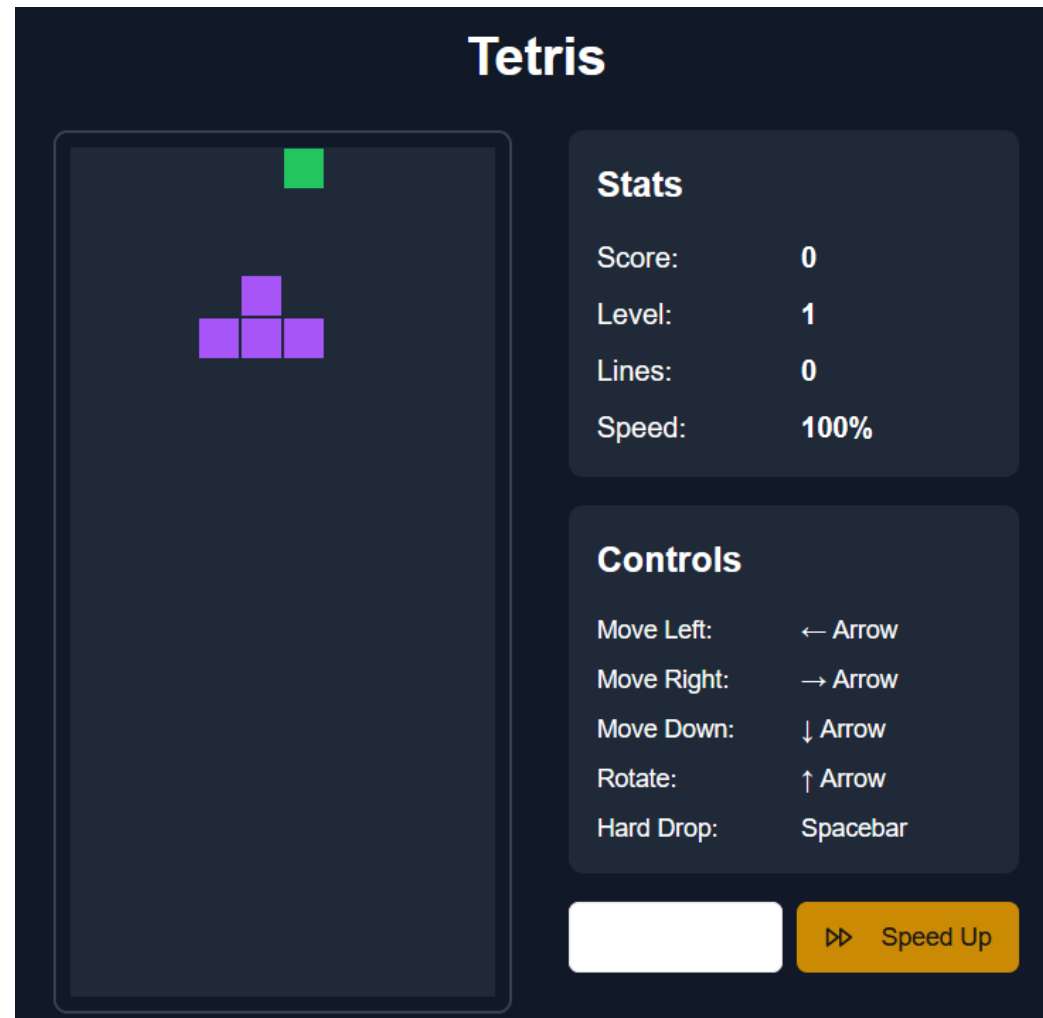
Bei Software-as-a-Service (SaaS) handelt sich um ein Cloud-basiertes Bereitstellungsmodell, bei dem Softwareanwendungen über das Internet als Dienst bereitgestellt werden – anstatt sie lokal auf dem Computer eines Nutzers zu installieren.

Beispiele: Outlook, Gmail, Jira, Slack, Trello, Github

Vibe Coding: Praxisbeispiel @leojr94_ 2/2



Vibe Coding: Praxisbeispiel v0-tetris



<https://v0-browser-tetris-game.vercel.app/>

Vibe Coding: Job-Angebot

Es sind bereits erste Job-Angebote als "AI Vibe Coder" aufgetaucht...

AI Vibe Coder

Beyond Chiefs [🔗](#)

20457 Hamburg
Hybrides Arbeiten

25 € pro Stunde - Werkstudent

Schnellbewerbung



Stellenbeschreibung

Gehalt

25 € pro Stunde

Anstellungsart

Werkstudent

Arbeitsort

20457 Hamburg

Leistungen

Aus der vollständigen Stellenbeschreibung

- Homeoffice-Möglichkeit

Vollständige Stellenbeschreibung

Role Overview: A Vibe Coder is a unique hybrid of Product Manager, Engineer, and UX thinker who designs experiences, questions roadmaps, and iterates until it feels right.

Core Responsibilities:

- **Building & Shipping:** Architect and launch end-to-end product features without bureaucracy
- **Design Through Code:** Create intuitive interfaces without being a formal designer
- **User-Centered Engineering:** Make technical and aesthetic decisions focused on user experience
- **Strategic Planning:** Own feedback collection and roadmap decisions without PM oversight
- **Solo Execution:** Build independently while knowing when to collaborate 1

Technical Requirements:

- Strong foundation in TypeScript and Python for AI development
- Experience with AI-first development tools (Cursor, bolt, n8n etc. AI-powered agents)
- Proficiency in strongly-typed languages

Quelle

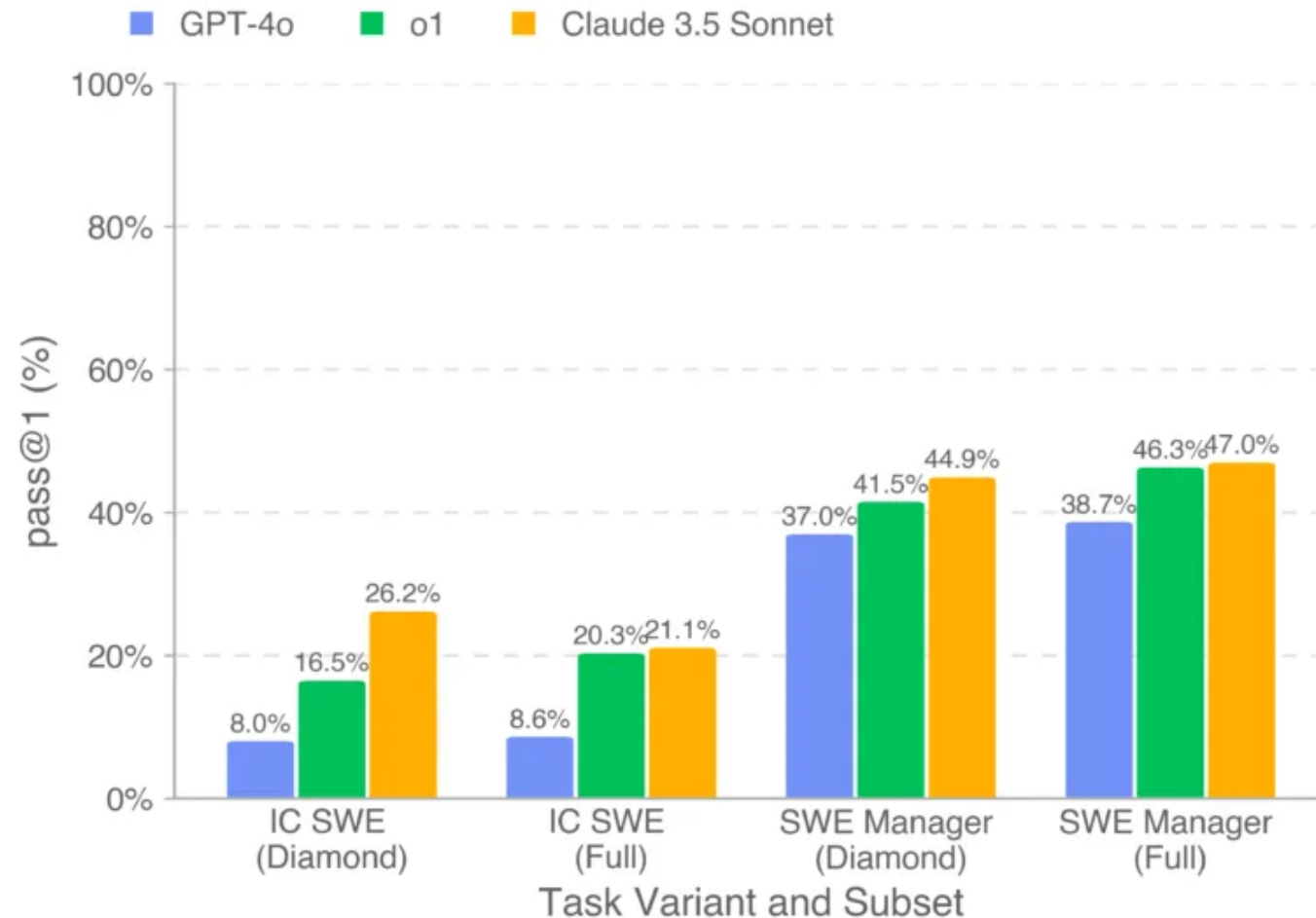
Vibe Coding: Tools (Auswahl)

- <https://www.cursor.com/>
- <https://windsurf.ai/org/de>
- <https://v0.dev/>
- <https://bolt.new/>
- <https://firebase.studio/>
- <https://github.com/openai/codex>
- <https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview>
- <https://www.jetbrains.com/junie/>
- <https://code.visualstudio.com/docs/copilot/chat/chat-agent-mode>
- <https://github.com/features/copilot>
- <https://superwhisper.com/> (speech to text)

Stand: AI in der Softwareentwicklung

"Ein neuer Benchmark von OpenAI zeigt, dass KI-Modelle im Praxistest zwar Potenzial zeigen, aber noch weit davon entfernt sind, komplexe Softwareprojekte eigenständig zu bewältigen."

"Eine detaillierte Analyse der Ergebnisse offenbarte eine typische Schwachstelle der KI: Die Modelle konnten zwar oft die fehlerhafte Stelle im Code lokalisieren, scheiterten aber häufig daran, die zugrundeliegende Ursache zu verstehen und eine umfassende Lösung zu entwickeln."



Quelle: <https://the-decoder.de/openai-testet-ki-als-softwareentwickler-fast-die-haelfte-des-code-budgets-geht-an-die-maschine/> (19. Februar 2025)

Model Collapse (AI-Degeneration)



"We find that indiscriminate use of **model-generated content in training causes irreversible defects in the resulting models**, in which tails of the original content distribution disappear. We refer to this effect as 'model collapse' and show that it can occur in LLMs"

"To sustain learning over a long period of time, we need to make sure that access to the **original data source is preserved and that further data not generated by LLMs remain available over time**. The need to **distinguish data generated by LLMs from other data** raises questions about the provenance [Herkunft] of content that is crawled from the Internet: it is unclear how content generated by LLMs can be tracked at scale."

AI in der Softwareentwicklung

Aufgrund von **Halluzinationen** und **Nicht-Determinismus** ist der Einsatz von AI in der Softwareentwicklung aktuell riskant.

Ein **möglicher Ansatz** zur Abmilderung der Risiken könnten **automatisierte Tests** darstellen, um zu prüfen, ob der generierte Code die **Anforderungen erfüllt**.

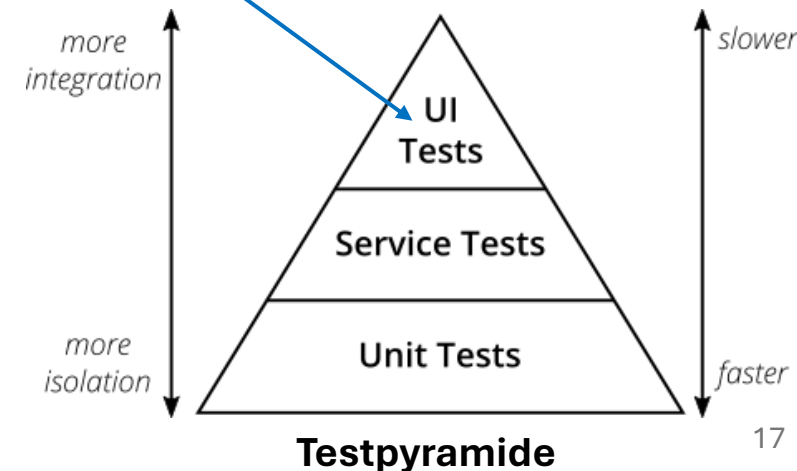


Exkurs: Akzeptanztest (Abnahmetest)



Akzeptanztest ist eine Teststufe in der Softwareentwicklung, bei der überprüft wird, **ob ein System die Anforderungen und Erwartungen des Kunden oder Nutzers erfüllt**. Es ist meist der letzte Test vor der Freigabe und Auslieferung der Software.

Akzeptanztests können **automatisiert** als **End-to-End-Test** mittels geeigneter Test-Frameworks umgesetzt werden.



Beispiel: Akzeptanztest als End-to-End-Test

Die komplette Anwendung wird hier aus Sicht des Endanwenders getestet. Die Anwendung wird dabei wie eine Blackbox behandelt – es werden Benutzereingaben simuliert und die Ausgabe überprüft, ohne zu betrachten, wie intern das Ergebnis zustande kommt.

@Test

```
void shouldAddBookToShoppingBasket() {  
    shopping.searchForBook("title: JUnit in Action");  
    shopping.selectBook("author: Catalin Tudos");  
    shopping.addSelectedItemToShoppingBasket();  
    List<String> basket = shopping.itemsInBasket();  
    assertTrue(basket.contains("item: JUnit in Action"));  
}
```

// more test cases ...

Zukunft der Softwareentwicklung mit AI (?)

Schritte:

1. Akzeptanztest (Anforderungen) als Unit Test Cases schreiben.
- 2. AI generiert anhand der Tests die Implementation.
3. Verifizieren der generierten Implementation mithilfe der Akzeptanztests.
4. Wiederhole Schritt 2 und 3, bis das Ergebnis passt.

Rolle verändert sich: Softwareentwickler → Produktmanager

Produktmanager sind für die Produktstrategie eines Produkts verantwortlich, **legen die Anforderungen fest** und verwalten die Veröffentlichung von Funktionen.

Natürliche vs Formale Sprachen

"In order to make machines significantly easier to use, it has been proposed (to try) to **design machines that we could instruct in our native tongues**. this would, admittedly, make the machines much more complicated, but, it was argued, by letting the machine carry a larger share of the burden, life would become easier for us. It sounds sensible provided you blame the obligation to use a formal symbolism as the source of your difficulties."

"The virtue of formal texts is that their manipulations, in order to be legitimate, need to satisfy only a few simple rules; they are, when you come to think of it, an amazingly effective tool for **ruling out all sorts of nonsense** that, when we use our **native tongues**, are **almost impossible to avoid**."

Quelle: On the foolishness of "natural language programming" – Edsger W.Dijkstra (1978)

<https://www.cs.utexas.edu/~EWD/transcriptions/EWD06xx/EWD667.html>